Screenshot Testing Strategy: BILD · Sport BILD · WELT News · WELT Edition

Goal: catch unintended visual changes on every pull request, on iOS and Android, by rendering real production UI with mocked data that is identical on every run. Ship faster with less manual visual regression testing.

Recommendation in one line: native, in-process snapshot tools on each platform (**swift-snapshot-testing** on iOS, **Roborazzi** on Android), fed by **one shared set of JSON fixtures**, gated in CI on every PR. A small nightly end-to-end screenshot pass covers what in-process rendering cannot see.

---

1. Tooling choice

	iOS	Android
PR gate (Layer 1)	swift-snapshot-testing 1.19, pinned exactly	Roborazzi 1.77 on Robolectric, pinned exactly
Renders	real SwiftUI/UIKit views, in the test process, on a pinned simulator	real Compose/View UI on the JVM (Robolectric Native Graphics), no emulator
Why	Same authors as TCA (the WELT iOS stack), so data mocking reuses `withDependencies`. Swift Testing support. Very widely adopted	Runs inside the normal `test` task and works with **Hilt** (fakes via `@TestInstallIn`). The same JVM run also drives **UI interaction tests** (tap, type, switch tabs). Creates `_compare.png` diff images. Used by Google's *Now in Android*
Nightly (Layer 2)	Maestro flows on simulator, app started with `-fixture <name>`	Maestro on emulator, `--es fixture <name>`

Alternatives considered:

- **Paparazzi** is the strongest Android alternative. It's 2.0 alpha right now, has no Hilt or interaction support, and needs its own Gradle task. It remains a cheap fallback, because our test code is a thin layer over the fixtures.
- **Google Compose Preview Screenshot Testing** is still alpha and requires AGP 9.
- **One cross-platform tool (Appium/Maestro + Applitools):** a single pipeline, but minutes per screen instead of milliseconds. It's flakier, it's paid per checkpoint, and edge states are hard to reach. It's the wrong economics for a PR gate, so it's used only for Layer 2.
- **Hosted review** (Sentry Snapshots, formerly Emerge): worth a phase-3 trial for review UX.

2. Mocked, deterministic data

One fixture catalog for both platforms. `/fixtures/feed/*.json` mirrors the API response. Each platform decodes it with its **production decoder**, so fixtures also test parsing. Examples: an unknown item type must be ignored, and `null` images must be handled. A fixture contract test on each platform fails if a fixture stops decoding. Snapshot names are identical across platforms (`test_feed_default.dark.png`), which gives designers a side-by-side gallery.

The network seam. UI and view models depend on a `FeedClient` value/interface, never on URLSession or OkHttp directly.

Seam	iOS (production codebase)	Android (production codebase)
Feed data	TCA `@DependencyClient` → `withDependencies { $0.feedClient = .fixture("feed_default") }`	Hilt: `@TestInstallIn` replaces `NetworkModule`, or `@BindValue` a fixture-backed repository
Images	Inject a Kingfisher/Nuke pipeline whose data loader reads `/fixtures/images` synchronously	Coil `FakeImageLoaderEngine`
Login	`AuthClient` value with an offline `.demo` backend	`AuthClient` interface with the same demo backend
Integration level	`URLProtocol` stub serving fixture files	OkHttp `MockWebServer` serving the same files

Everything else that varies is pinned too:

- clock (`now = 2026-11-05T14:13:20Z`, the day the demo's real headlines were fetched), locale `de_DE`, time zone `Europe/Berlin`
- randomness: games deal from a **seeded generator implemented identically on both platforms**, so a seed gives the same board on iOS and Android (asserted by one shared literal)
- animations off (the live-ticker pulse has a switch)
- fixed device geometry, theme and text size for each variant
- ads and WebViews replaced by fixed-size placeholders, so third-party SDKs never render in tests

Error and offline states are injected as client failures (`FeedError.offline`, `.server(503)`), not as data.

Real content, legally: fixtures use real public headlines and links (WELT/BILD RSS) with our own one-line teasers. Article text is never copied: tapping opens the publisher's page in an in-app browser. A contract test asserts every item links out over HTTPS and credits its source.

3. Golden images: storage and review

- **Storage:** goldens live next to the tests and move to **Git LFS** past roughly 100 MB, so they're reviewed with the code that changed them.
- **Who may create goldens:** only CI, on the pinned runner image. Developers add the `record-snapshots` label, CI re-records and pushes a bot commit, and the golden diff appears in the PR. This removes "works on my Mac" goldens entirely.
- **Review:** on failure, CI uploads a diff report (golden | this PR | changed pixels, plus an HTML gallery) and comments on the PR. `CODEOWNERS` requires **design or QA approval for any golden change**, which is what stops "re-record and merge" from turning the tests into a rubber stamp.
- **Matrix (curated, not every combination):**
  - every component: light + dark
  - text-heavy components: + accessibility size AX3 (iOS) / font scale 2.0, Android's maximum. AX5 is deliberately left to Layer 2: running the app at AX5 shows the title hyphenating ("NewsDe-mo"), a cost/coverage trade-off worth revisiting
  - screens: + smallest phone
  - premium/paywall: both themes, always

4. CI/CD and the Scrum process

- **Every PR:** iOS (macOS runner) and Android (Linux runner) jobs run in parallel. Target time is under 10 minutes. In the demo, 62 snapshots per platform run in about 30 s (iOS) and 12 s (Android) after the build.
- **Rollout of the gate:** report-only at first, then a required check. At scale only affected modules run (Gradle's task graph on Android; per-module test plans or Tuist selective testing on iOS).
- **Nightly:** Layer 2 Maestro flows plus a full matrix run. Results go to the QA channel.
- **Release branch:** the same gate. Manual visual regression testing shrinks to exploratory testing.
- **Definition of Done:** a new UI state ships with a fixture and a snapshot. Designers review goldens in the PR, instead of in a pre-release screenshot session.
- **Platform upgrades** (Xcode, simulator runtime, AGP, Robolectric, snapshot library): one PR that bumps the version *and* re-baselines all goldens, reviewed as such.

5. Stability (flakiness)

Snapshot tests should be deterministic by construction rather than retried until green.

1. **Pin the environment:** Xcode version, simulator device and runtime, runner image, library versions. During this demo the iOS 27 simulator runtime disappeared mid-session. The pin made the cause obvious, and the fix was one re-baseline commit.
2. **Pin the inputs:** see §2. Any `Date()`, network image, animation or random value in a view is a test-design bug.
3. **Explicit, minimal tolerance:**
   - iOS uses `precision 1.0` (every pixel compared) with `perceptualPrecision 0.98`, which absorbs GPU anti-aliasing noise but fails on a 2pt padding shift (proven in the regression demo).
   - Android uses zero tolerance (software rendering is identical run to run).

    - The tolerance is never loosened globally.
4. **No automatic retries for snapshots.** A flaky snapshot is quarantined within 24 hours with an owner and a fix-by date. The flake rate is tracked, with a target below 1%.
5. **The harness is tested too.** Components taller than the screen were silently clipped; the fix was to measure and host them without safe areas.
6. **A green test can protect nothing.** After moving Android to SDK 37, **44 of 62 captures were blank** on Robolectric's SDK 37 runtime, and verify still passed, because blank equals blank. The fixes:
    - pin the test runtime to SDK 36 (the app still targets 37)
    - a capture guard that refuses to record or compare an undrawn frame
    - `make lint-goldens`, a CI job that fails on any blank golden
7. **Pin test dependencies, not just app dependencies.** Compose UI tests pulled in Espresso 3.5.1 transitively, which calls an API removed in SDK 37, so every UI test crashed. Pinning Espresso 3.7.0 fixed it.
8. **What can't be captured can't be protected.** iOS 26's Liquid Glass tab bar renders blank in in-process snapshots (an open upstream question). The app therefore uses its own floating bar, identical on both platforms, with accessibility implemented explicitly. System-bar fidelity belongs to Layer 2.
9. **Clear old diff output before every run.** Stale `_compare.png` files from a previous run showed up next to the real failures and would have misled a reviewer.

6. Integration into the existing production codebase

- **Fixtures** live in one shared `news-fixtures` package: an SPM resource package plus a Gradle artifact, versioned with the API contract and validated against the OpenAPI schema. They are seeded from real, anonymised API responses by a recording script, then curated by hand for edge cases.
- **Start where there are no data dependencies:** design-system components first, then feature modules one at a time, opt-in. No big-bang refactor; adding the `FeedClient`-style seam is the same refactor that makes view models unit-testable.
- **Snapshot support code** (the `deterministic()` helper, variants, the diff report) is a small internal test-support module on each platform, so each new feature test is about 5 lines.

7. Scaling as apps and team grow

- **Four apps:** one design system means component goldens are shared. App-specific modules (WELT premium, Sport BILD tickers) own their own screens and fixtures.
- **Cost control:** run only affected modules, shard on CI, use Git LFS, and review the matrix every quarter (remove variants that never fail).
- **Ownership:** developers write and maintain the tests. QA owns the strategy, the fixture catalog and the matrix rules. Design owns golden approval.
- **Enablement (how QA mentors the team):** a one-page "add a snapshot in 5 lines" guide, pairing with each squad in sprint 2, a short diff-review session for designers, office hours during the first month, and fixture review as part of story refinement ("which states does this story add?").
- **Metrics:** escaped visual bugs per release, PR gate time, flake rate, and the share of UI states with fixtures.

8. Rollout plan (about 4 sprints, non-disruptive)

Sprint	Step	Exit criterion
1	Spike on one app (e.g. WELT News iOS + Android): support module, 5 design-system components, CI job **report-only**	Green for 5 days with no flakes
2	Shared `news-fixtures`, feed screen states, diff-report PR comments, CODEOWNERS; pairing sessions with each squad	Every squad has merged at least one snapshot test
3	Make the gate **required**, add to the Definition of Done, record goldens on CI only; start the Layer 2 nightly	Under 10 min PR time, flake rate under 1%
4+	Roll out to the other 3 apps; evaluate a hosted review tool; quarterly matrix review	Visual bugs found by the gate, not in release testing

9. Devil's advocate: risks and answers

Objection	Answer
"Snapshots only detect change. Reviewers will approve without looking."	Small, focused snapshots; mandatory design/QA approval; the PR comment shows the diff, so nobody has to dig for it
"In-process rendering isn't the real app (system bars, navigation, WebViews)."	True. That's why Layer 2 exists. Layer 1 buys speed and coverage of every state
"Every OS or Xcode upgrade changes every golden."	Yes. Planned, as a single re-baseline PR per upgrade
"Too many combinations, so slow CI and review fatigue."	Curated matrix rules, affected-modules-only runs, quarterly pruning
"Goldens bloat the repo."	Git LFS; goldens are only ever written by CI
"Roborazzi is largely one maintainer."	Real risk. Mitigated by exact pinning and thin test code on top of portable fixtures, so switching to Paparazzi is cheap
"QA should own the tests."	No. Developers own them (shift-left). QA owns the system around them
"AI could approve diffs."	AI can *triage* (summarise the diff in the PR comment, flag likely-intended changes, generate edge-case fixtures like long German compound words). A human always approves goldens

What the demo caught before any deliberate regression (product issues; tooling issues are in §5):

1. The breaking-banner timestamp was grey on red in light mode (a contrast bug).
2. The kicker was truncated next to the PLUS badge at accessibility text sizes.
3. "EILMELDUNG" was hyphenated at the same text sizes.
4. **Cross-platform divergence:** Android didn't hyphenate German headlines (Compose defaults to `Hyphens.None`) while iOS did. This was visible only because both platforms render the same fixture.
5. A 60-letter compound word still breaks mid-word on **both** platforms. This was handed over as a **content finding**: the fix is soft hyphens from the CMS, not UI code.
6. **Found only by running the real app (Layer 2), never by snapshots:** on an en_US device the 12-hour preference leaked into German dates ("11:05 PM Uhr"). Snapshots pin `de_DE`, so they can't see it. A pinned environment proves the UI is *stable*, not that it's *right everywhere*. That's why Layer 2 exists, and why the matrix should carry one "hostile device settings" variant.
7. The login e-mail placeholder rendered as a **blue link**: SwiftUI parsed the string as Markdown.

The deliberate regression (a "refactor" that drops the PLUS badge and tightens the banner by 2pt) then failed **the same 19 of 62 snapshots on both platforms**. That includes a real WELTplus teaser losing its paywall badge, while every unaffected screen stayed green (docs/regression).